YACHAY TECH UNIVERSITY

//01

Divide and Conquer & Master Theorem

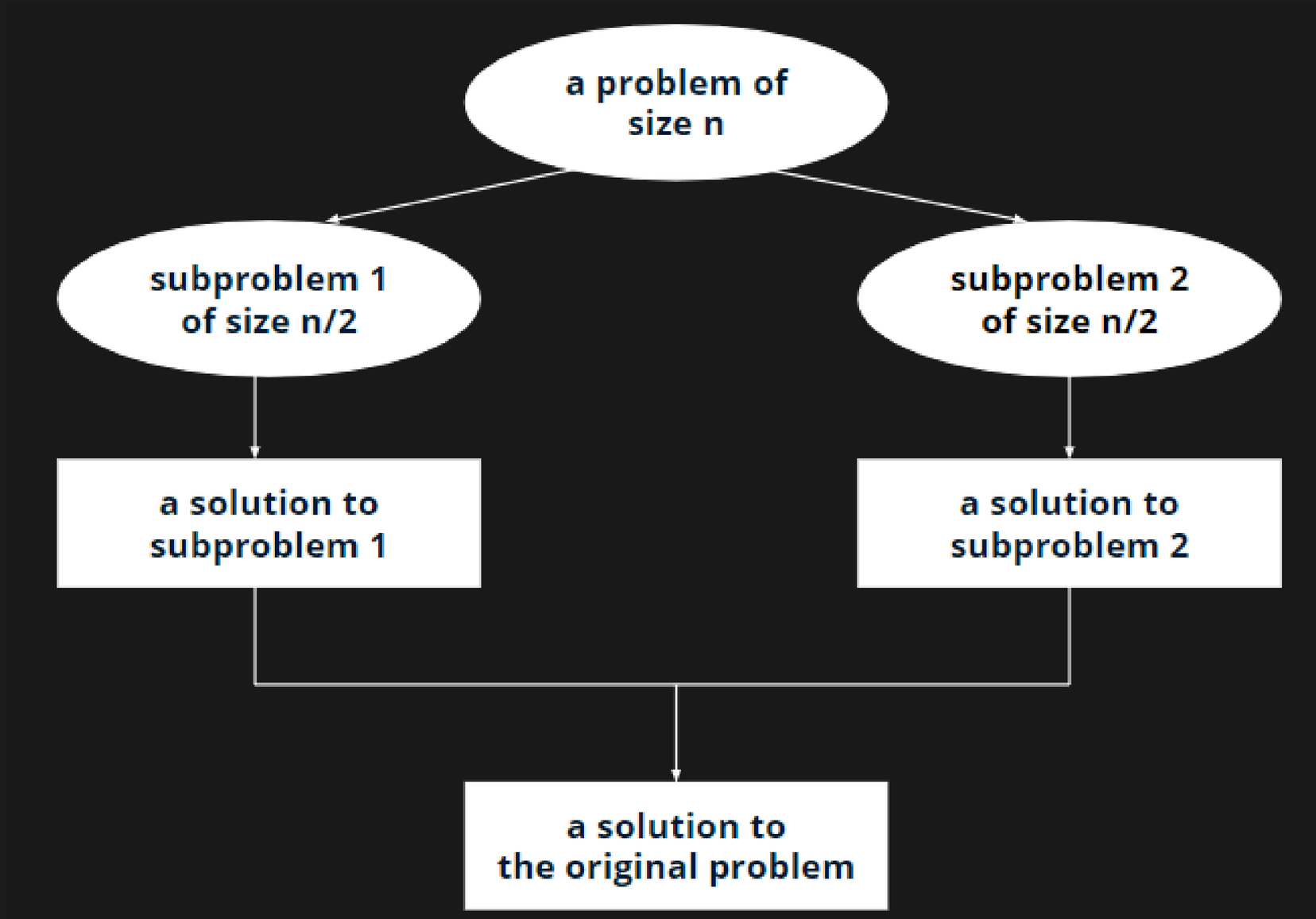
Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Divide-and-Conquer

The most well known algorithm design strategy:

1. Divide instance of problem into two or more smaller instances
2. Solve smaller instances recursively
3. Obtain solution to the original (larger) instance by combining these solutions

Divide-and-Conquer



Divide-and-Conquer Examples

- Sorting: mergesort and quicksort
- Binary tree traversals
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and convex-hull algorithms

- Binary search: decrease-by-half (or degenerate divide&conquer.)

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem:

If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Master Theorem Examples

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem:

If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

$T(n) = 4T(n/2) + n \Rightarrow T(n) \in ?$

$T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in ?$

$T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in ?$

Merge Sort and Quicksort

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Mergesort

- Split array A[0..n-1] in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
  - Repeat the following until no elements remain in one of the arrays:
    - compare the first elements in the remaining unprocessed portions of the arrays
    - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
  - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Pseudocode of Mergesort

ALGORITHM *Mergesort*$(A[0..n-1])$

//Sorts array $A[0..n-1]$ by recursive mergesort

//Input: An array $A[0..n-1]$ of orderable elements

//Output: Array $A[0..n-1]$ sorted in nondecreasing order

if $n > 1$

copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort$(B[0..\lfloor n/2 \rfloor - 1])$

Mergesort$(C[0..\lceil n/2 \rceil - 1])$

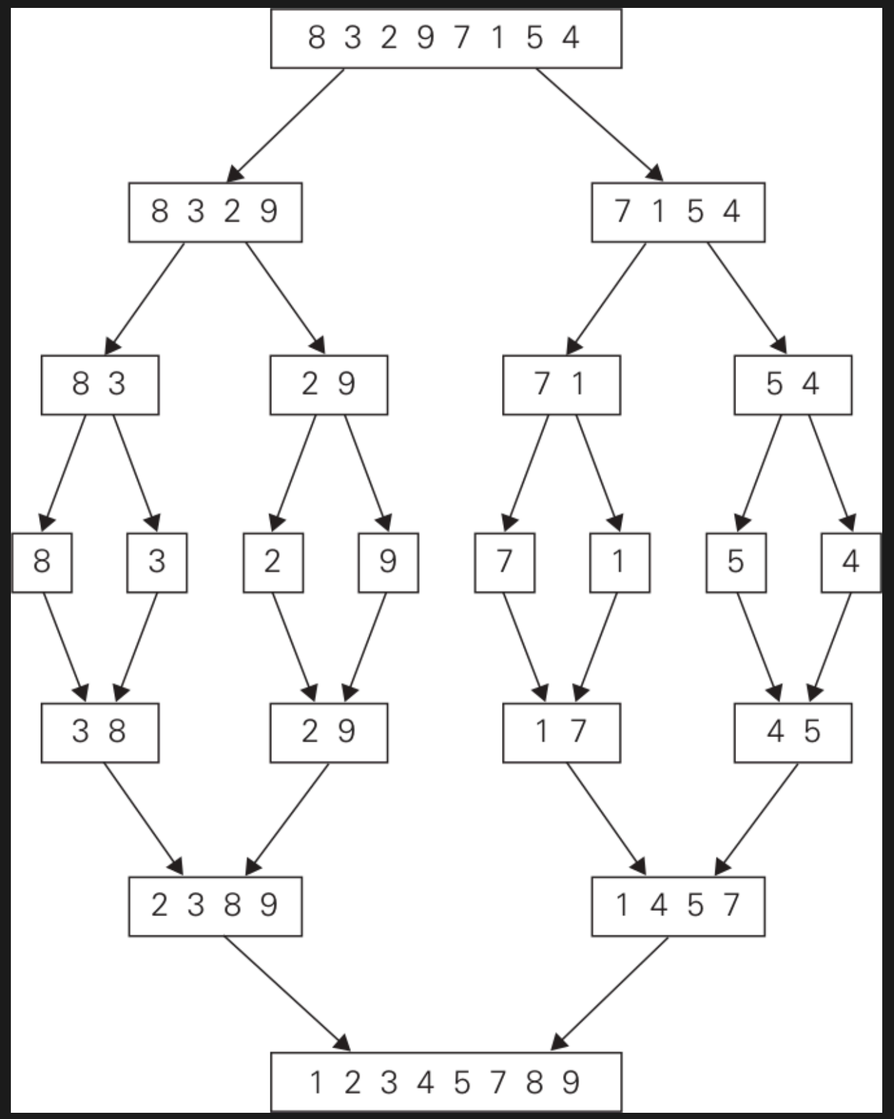
Merge(B, C, A)

Pseudocode of Merge

ALGORITHM $Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])$

```
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted
//Output: Sorted array A[0..p + q − 1] of the elements of B and C
i ← 0; j ← 0; k ← 0
while i < p and j < q do
    if B[i] ≤ C[j]
        A[k] ← B[i]; i ← i + 1
    else A[k] ← C[j]; j ← j + 1
    k ← k + 1
if i = p
    copy C[j..q − 1] to A[k..p + q − 1]
else copy B[i..p − 1] to A[k..p + q − 1]
```

Mergesort Example



Analysis of Mergesort

All cases have same efficiency: $\Theta(n \log n)$

Number of comparisons in the worst case is close to theoretical minimum for comparison-based sorting:

$$\lceil \log_2 n! \rceil \approx n \log_2 n$$

Space requirement: $\Theta(n)$ (not in-place)

Can be implemented without recursion (bottom-up)

Quicksort

- Select a pivot (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining n-s positions are larger than or equal to the pivot (see next slide for an algorithm)

- Exchange the pivot with the last element in the first (i.e., ≤) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

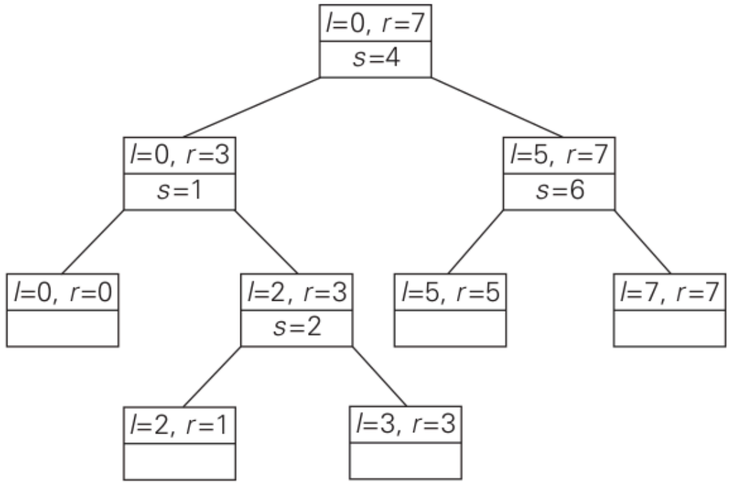
Hoare's Partitioning Algorithm

```
Algorithm Partition(A[l..r])
//Partitions a subarray by using its first element as a pivot
//Input: A subarray A[l..r] of A[0..n − 1], defined by its left and right
//       indices l and r (l < r)
//Output: A partition of A[l..r], with the split position returned as
//        this function's value
p ← A[l]
i ← l;   j ← r + 1
repeat
    repeat i ← i + 1 until A[i] ≥ p
    repeat j ← j − 1 until A[j] · p
    swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j])   //undo last swap when i ≥ j
swap(A[l], A[j])
return j
```

Quicksort Example

5 3 1 9 8 2 4 7

0	1	2	3	4	5	6	7
	i						*j*
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	2	8	9	7
				j	*i*		
5	3	1	4	2	8	9	7
2	3	1	4	**5**	8	9	7
	i		*j*				
2	3	1	4				
	i	*j*					
2	3	1	4				
	i	*j*					
2	1	3	4				
	j	*i*					
2	1	3	4				
1	**2**	3	4				
1							
			i j				
		3	4				
		j	*i*				
		3	4				
			4				
						i	*j*
					8	9	7
						i	*j*
					8	7	9
						j	*i*
					8	7	9
					7	**8**	9
					7		
							9



(b)

Analysis of Quicksort

Best case: split in the middle — $\Theta(n \log n)$
Worst case: sorted array! — $\Theta(n^2)$
Average case: random arrays — $\Theta(n \log n)$

Improvements:

- better pivot selection: median of three partitioning
- switch to insertion sort on small subfiles
- elimination of recursion

These combine to 20-25% improvement

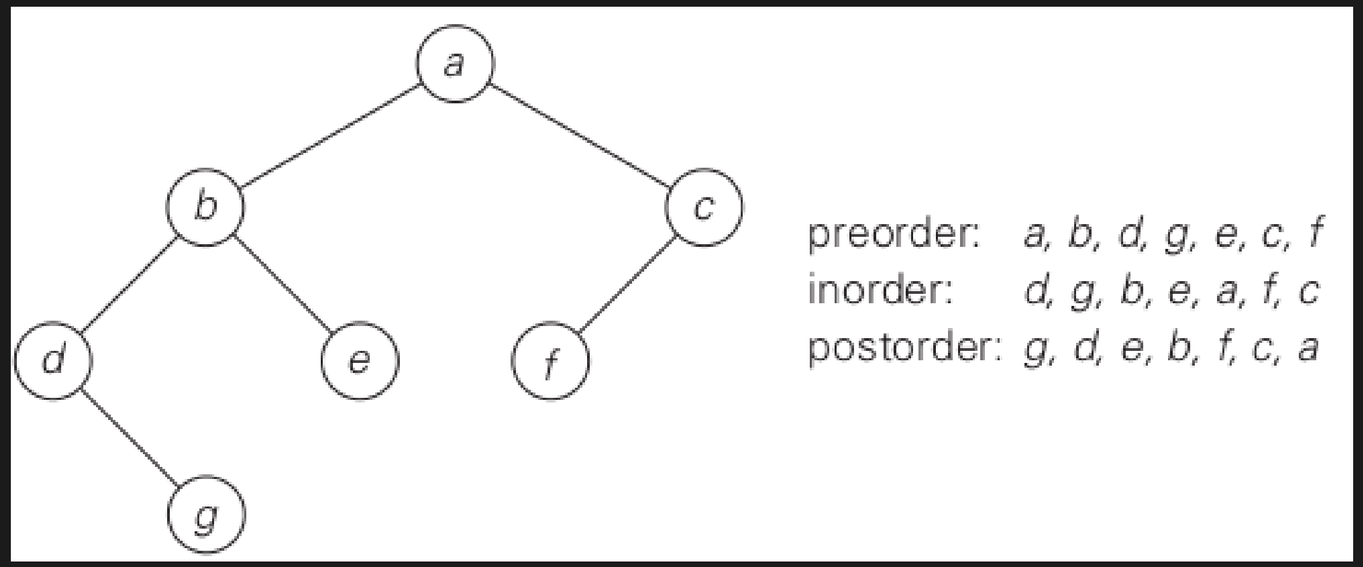
Considered the method of choice for internal sorting of large files ($n \geq 10000$)

Binary Tree Algorithms

Binary tree is a divide-and-conquer ready structure!

Ex. 1: Classic traversals (preorder, inorder, postorder)

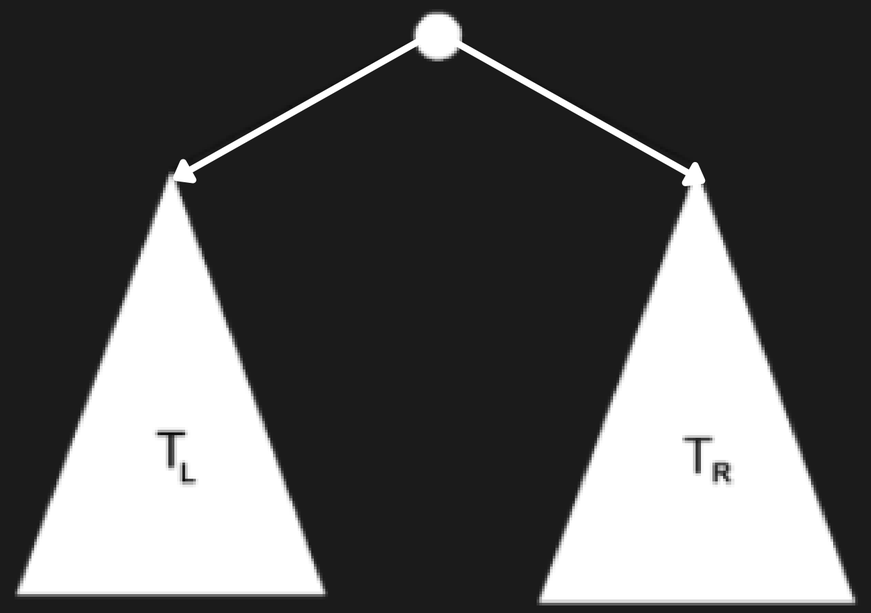
```
Algorithm Inorder(T):
    if T ≠ ∅
        Inorder(T_left )
        print(root of T)
        Inorder(T_right )
```



Efficiency: $\Theta(n)$

Binary Tree Algorithms (cont.)

Ex. 2: Computing the height of a binary tree



$h(T) = \max\{h(T_L), h(T_R)\} + 1$ if $T \neq \varnothing$ and $h(\varnothing) = -1$

Efficiency: $\Theta(n)$

Q & A

EMAIL ADDRESS — ipineda@yachaytech.edu.ec

TUTORING — https://calendly.com/ipineda

WEBSITE — www.israelpineda.com
www.yachay-scg.com

FOR QUESTIONS OR COMMENTS